Game Engine Architectures 04

C++, AI Agents, Animations

Programming in C++

- If you want to program in C++, you need to either choose a C++ project in the beginning, or you can create a C++ Class to let UE do the setup for you later (+ re-generating the project files)
- Usually, we then start the UE Project from the programming IDE instead of from the launcher
- We can then access all C++ files and functions and recompile on changes
- Rule of thumb: When changing the header file, it's best to restart the project instead




```
#include "SlendermanCharacter.h"

// Sets default values
ASlendermanCharacter::ASlendermanCharacter()
{
    // Set this character to call Tick() every frame.
    // You can turn this off to improve performance if you don't need it.
    PrimaryActorTick.bCanEverTick = true;

}

// Called when the game starts or when spawned
void ASlendermanCharacter::BeginPlay()
{
    Super::BeginPlay();

}

// Called every frame
void ASlendermanCharacter::Tick(float DeltaTime)
{
    Super::Tick(DeltaTime);

}

// Called to bind functionality to input
void ASlendermanCharacter::SetupPlayerInputComponent(UInputComponent* PlayerInputComponent)
{
    Super::SetupPlayerInputComponent(PlayerInputComponent);

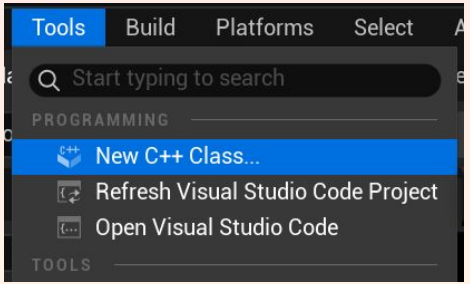
}
```

Blueprints and C++ don't substitute each other, they're complimentary

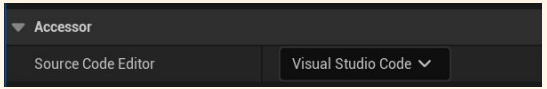
Creating your first C++ Class

- creating a C++ class offers similar options as creating a BP class
- you can choose your parent class from any existing C++ class (Actor, AIController, ActorComponent, Subsystem, ...)
- UE automatically creates a *.cpp* and *.h* file with a generic structure

create a new class, refresh your project files (in case of changes like version updates), or open your programming IDE from here



change the IDE in Editor *Preferences*



.h vs *.cpp* Recap

Header File *.h*

- contains C++ code
- used for templating and class definitions
- we *define* our functions and variables in here
- we can then include this file in source files to access the Class' functions and variables

Source File *.cpp*

- contains C++ code
- used for implementations and source code
- we *implement* our functions and variables in here
- it's possible to mix the two, but generally bad practice



… in Unreal Engine

- UE automatically creates a *.cpp* and *.h* file with a generic structure when creating a class
- our header file defines our functions, variables and parent class
- the header file also defines which variables and functions are visible and/or editable in the Editor, as well as reachable in BPs

```
#pragma once

#include "CoreMinimal.h"
#include "AIController.h"
#include "EnemyStateTypes.h"
#include "SlendermanCharacter.h"
#include "SlendermanAIController.generated.h"

UCLASS()
class SLENDERMANPROJECT_API ASlendermanAIController : public AAIController
{
    GENERATED_BODY()


public:
    virtual void Tick(float DeltaSeconds) override;


protected:
    virtual void OnPossess(APawn* InPawn) override;
    virtual void OnMoveCompleted(FAIRequestID RequestID, const FPathFollowingResult& Result) override;

    UPROPERTY(EditAnywhere, BlueprintReadOnly, Category = "Slenderman")
    float PatrolAcceptanceRadius = 50.0f;


public:
    UFUNCTION(BlueprintCallable, Category = "Slenderman Actions")
    void UpdateState(EEnemyState NewState);

    UFUNCTION(BlueprintCallable, Category = "Slenderman Actions")
    void FindNextPatrolPoint();


private:
    ASlendermanCharacter* Slenderman;
    APawn* Player;

    TArray<FVector> PatrolPoints;
    int32 PatrolIndex = 0;

    void RestartLevel();

    FTimerHandle RestartHandle;
    FTimerHandle PatrolWaitHandle;

};
```

the .generated.h include needs to be the last one!

UE Macros

UE C++ Conventions

- UE has a specific Prefix-Convention that you **should** follow
- The compiler also recognizes the type based on the name
- Also, then it's easier to recognize the type for you as the programmer as well!
- Unreal Directive: https://unrealdirective.com/resources/project-standards/cpp-standards/naming/

Type prefixes

PREFIX	USE	EXAMPLES
A	`AActor` descendants	`ACharacter`, `AProjectile`
U	`UObject` descendants that aren't actors	`UActorComponent`, `UItemDefinition`
F	Structs and most plain C++ types	`FVector`, `FInventoryEntry` *also Delegates!
E	Enums	`ETeamAttitude`, `EWeaponState`
I	The native half of an Unreal interface	`IInteractable`
S	Slate widgets	`SCompoundWidget`
T	Template types	`TArray`, `TSharedPtr`

UE adds the prefix automatically to the name when creating a Class!

Constructor, BeginPlay, Tick

- these are all accessible in the C++ file as well
- UE usually sets them up automatically when creating a child of an Actor
- the constructor is similar to the Construction Script in a BP

```cpp
#include "SlendermanCharacter.h"

// Sets default values
ASlendermanCharacter::ASlendermanCharacter()
{
    // Set this character to call Tick() every frame.
    // You can turn this off to improve performance if you don't need it.
    PrimaryActorTick.bCanEverTick = true;

}

// Called when the game starts or when spawned
void ASlendermanCharacter::BeginPlay()
{
    Super::BeginPlay();

}

// Called every frame
void ASlendermanCharacter::Tick(float DeltaTime)
{
    Super::Tick(DeltaTime);

}

// Called to bind functionality to input
void ASlendermanCharacter::SetupPlayerInputComponent(UInputComponent* PlayerInputComponent)
{
    Super::SetupPlayerInputComponent(PlayerInputComponent);

}
```

Macros and their Specifiers

- there are a few different Macros you can use, and hundreds of different specifiers
- in the beginning, you'll mostly use *UPROPERTY* and *UFUNCTION*
- these define which variables, and how they are accessible in the Editor, and also how they look like
- they also define what type of definition this is (class, enum, ...)

the most common ones

Details Panel | BP Graph | Context Menu

```
UPROPERTY(EditAnywhere, BlueprintReadOnly, Category = "Slenderman")
EEnemyState CurrentState = EEnemyState::Patrol;

UPROPERTY(EditAnywhere, BlueprintReadWrite, Category = "Slenderman")
float SightRadius = 2000.0f;

UPROPERTY(VisibleAnywhere, BlueprintReadOnly, Category = "Slenderman")
float CatchRadius = 200.0f;

UPROPERTY(EditAnywhere, Category = "Slenderman")
float WaitTimeAtPoint = 5.0f;

UFUNCTION(BlueprintCallable, Category = "Slenderman Actions")
void UpdateState(EEnemyState NewState);

UFUNCTION(BlueprintCallable, BlueprintNativeEvent, Category = "Slenderman Actions")
void FindNextPatrolPoint();

UFUNCTION(BlueprintCallable, BlueprintPure, Category = "Slenderman Actions")
void EvaluatePlayer();
```

Explanation of all Macro Specifiers:
https://unreal-garden.com/

- UPROPERTY
- UFUNCTION
- UCLASS
- USTRUCT
- UINTERFACE
- UENUM and UMETA
- UPARAM

Everyday Functions #1

Player && World

```
#include "Kismet/GameplayStatics.h"

APawn* Player = UGameplayStatics::GetPlayerPawn(GetWorld(), 0);
APlayerController* PC = UGameplayStatics::GetPlayerController(GetWorld(), 0);

TArray<AActor*> Found;
UGameplayStatics::GetAllActorsWithTag(GetWorld(), FName("PatrolPoint"), Found);
// Caution: this scans EVERY actor in the world. Fine in e.g. BeginPlay, but not in Tick!

AActor* NewThing = GetWorld()->SpawnActor<AMyActor>(AMyActor::StaticClass(), Location, Rotation);
```

Checking Actors

```
// Cast is nullptr on failure
if (ASlendermanCharacter* Slender = Cast<ASlendermanCharacter>(GetPawn()))
{
    Slender->CurrentState = EEnemyState::Patrol;
}

if (SomeActor->IsA(ASlendermanCharacter::StaticClass())) { /* do sth */ }

FVector Pos = SomeActor->GetActorLocation();
SomeActor->SetActorLocation(NewPos);
```

Everyday Functions #2

Math

```
float Distance = FVector::Dist(MyPos, TargetPos);
FVector Dir = (TargetPos - MyPos).GetSafeNormal();

float Clamped = FMath::Clamp(Value, 0.f, 1.f);
float Lerped = FMath::Lerp(From, To, 0.5f);
int Rand = FMath::RandRange(0, NumPoints - 1);
```

Time Stuff

```
// put this in the header
FTimerHandle WaitHandle;

// handle, class, function to call on run out, time, loop
GetWorldTimerManager().SetTimer(WaitHandle, this, &AMyClass::MyFunction, 3.0f, false);
// this one loops, for example
GetWorldTimerManager().SetTimer(WaitHandle, this, &AMyClass::MyFunction, 2.0f, true);

// to cancel a timer; save to use even if timer is not running
GetWorldTimerManager().ClearTimer(WaitHandle);

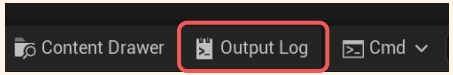
// seconds since world start
float Now = GetWorld()->GetTimeSeconds();
float DeltaTime  = GetWorld()->GetDeltaTimeSeconds();
```

Logging

Extra:

```
DrawDebugSphere(GetWorld(), Center, 200.f, 12, FColor::Red, false, 5.f);
DrawDebugLine(GetWorld(), From, To, FColor::Green, false, 5.f);
```

- UE unfortunately makes it a bit difficult to Debug Log in C++
- You have the option to print to the Debug Console, or to print on the screen (as is the default for BP's *Print String*)
- The function call needs a few different arguments (see image)



Log Category

Verbosity

```
UE_LOG(LogTemp, Warning, TEXT("Test swjklfekwbef"));
UE_LOG(LogTemp, Warning, TEXT("I'm printing a string var now: %s"), *Slenderman->GetName());
UE_LOG(LogTemp, Warning, TEXT("Wow! A float %f and an integer %d!"), 3.5f, 1);

if (GEngine)
{
    GEngine->AddOnScreenDebugMessage(-1, 5.0f, FColor::White, TEXT("You should see this on-screen"));
}
```

unique integer

Time

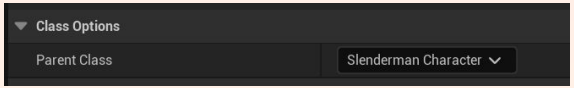
Colour

later on, it's good practice to define your own Log Category

Documentation

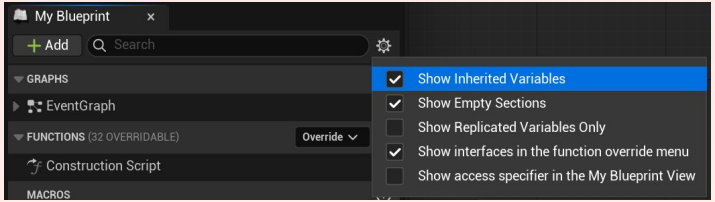
BP Children of C++ Classes

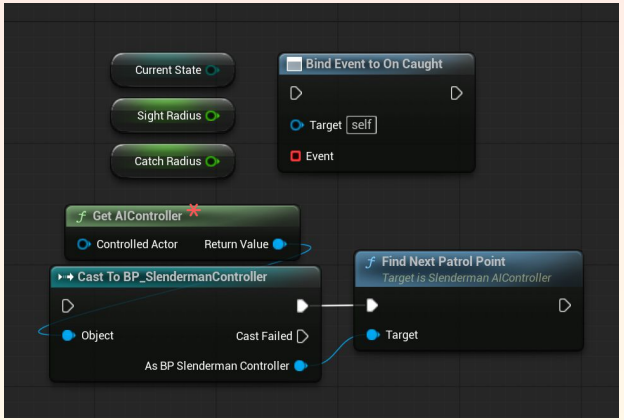
- you can choose your own C++ Class as the Parent Class when creating a BP Class
- you can also change this later on in the *Class Settings*



- You can then access all exposed variables, functions and events depending on the specifications you set in the header file

to see inherited Variables:



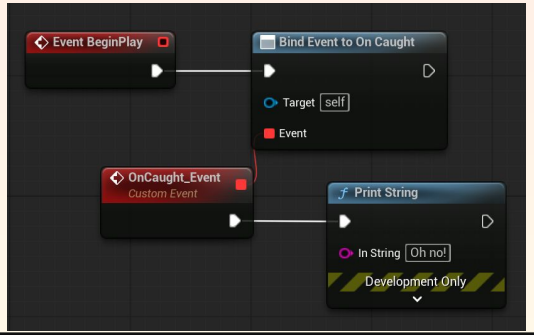


*note: don't actually do this inside your Pawn BP!
We don't want circular references

Events aka *Delegates*

- You can create Delegates with up to 5 parameters
- define them in your header file and broadcast them at positions in your source file

can be accessed in your BP child class



needs to be placed before the class definition in the header file!

```
DECLARE_DYNAMIC_MULTICAST_DELEGATE(FOnCaught);
DECLARE_DYNAMIC_MULTICAST_DELEGATE_OneParam(FOnDamageTaken, float, DamageAmount)
```

```
// Delegates
UPROPERTY(BlueprintAssignable, Category = "Slenderman Delegates")
FOnCaught OnCaught;
UPROPERTY(BlueprintAssignable, Category = "Slenderman Delegates")
FOnDamageTaken OnCaught;
```

inside the class in the header file

```
OnCaught.Broadcast();
OnDamageTaken.Broadcast(30.0f);
```

call inside a function

Delegates
Multicast Delegates (what we use for Events)

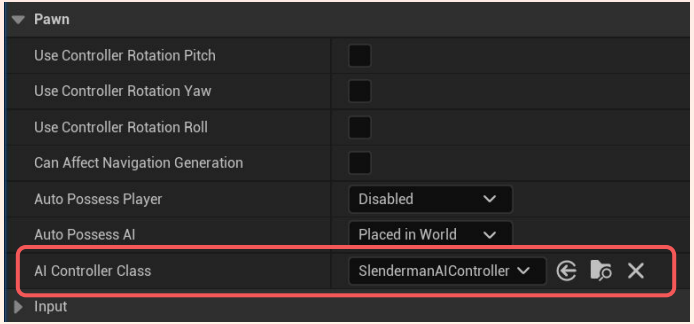
AI Navigation in UE

Pawn and AIController

- An AI Agent is made up of two components:
- **Pawn** (the visible thing, what we're controlling)
- **AIController** (the brain, making decisions)
- You can create your own (BP or C++ Class), inheriting from the parent classes *Pawn* and *AIController*



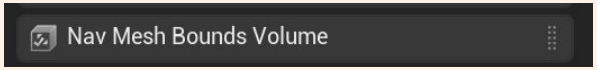
Inside your Pawn Blueprint, you can then define the dedicated AIController:

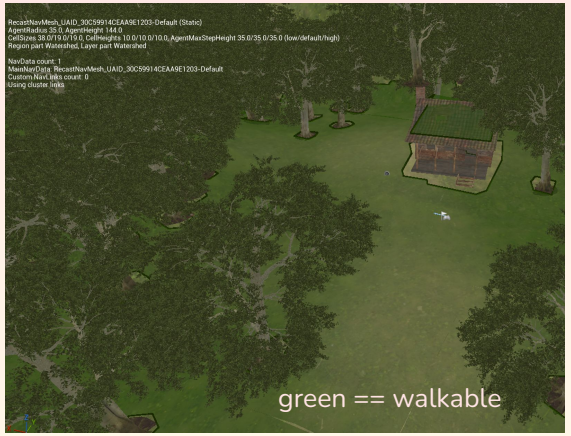


Place the Pawn in the Level and the controller should automatically possess it

Adding a Nav Mesh Volume

- Add a “*Nav Mesh Bounds Volume*” and adjust its size to cover your intended Area
- it will automatically calculate the places AI Agents can go
- Press “P” to visualise this





you can pre-build the navigation through *Build > Build Paths*

AIController in C++

- When inheriting from the AIController Class, we have the possibility to override specific functions like *OnPossess* or *OnMoveCompleted*
- We can also easily tell the AI Agent to move to a specific location or actor

```
UCLASS()
class SLENDERMANPROJECT_API ASlendermanAIController : public AAIController
{
    GENERATED_BODY()

protected:
    virtual void OnPossess(APawn* InPawn) override;
    virtual void OnMoveCompleted(FAIRequestID RequestID, const FPathFollowingResult& Result) override;
```

```
void ASlendermanAIController::OnMoveCompleted(FAIRequestID RequestID,
        const FPathFollowingResult& Result)
{
    Super::OnMoveCompleted(RequestID, Result);

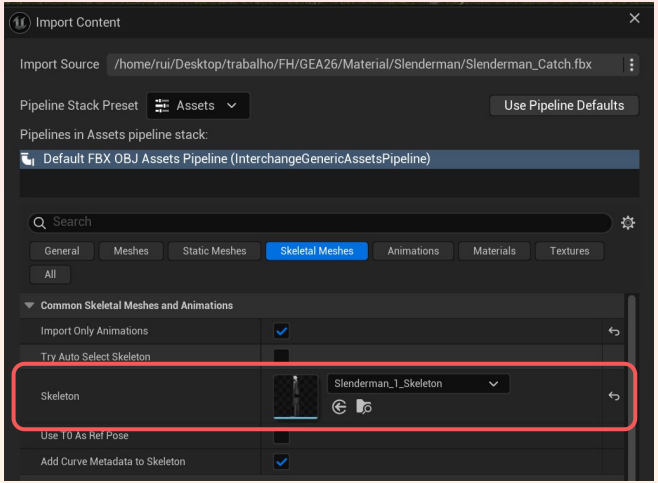
    if (Result.IsSuccess())
    {
        MoveToLocation(PatrolPoints[PatrolIndex], PatrolAcceptanceRadius);
        MoveToActor(Player, 50.0f);
    }
}
```

Animation Blueprints

Downloading and Importing Animations

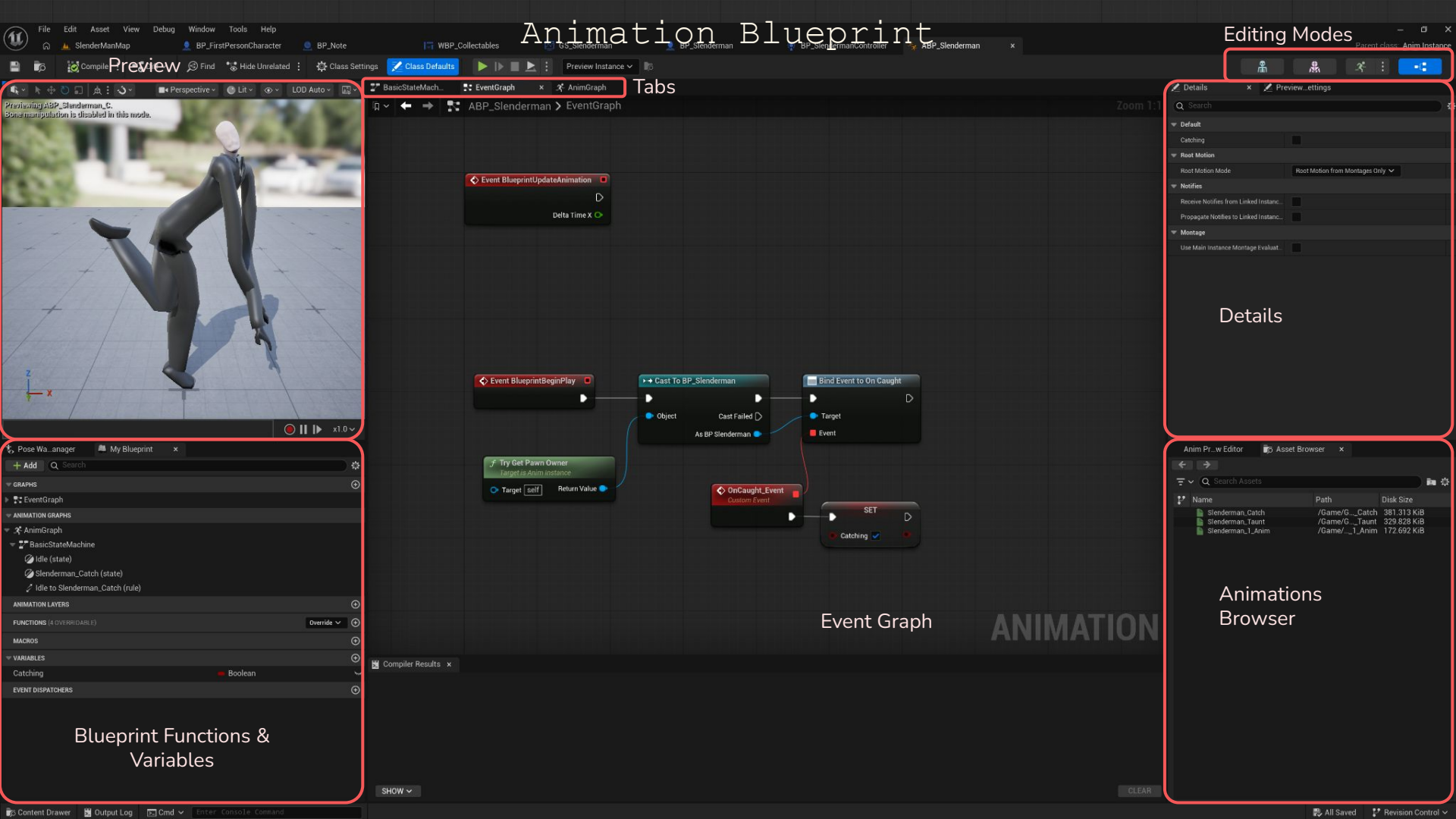
- If you want to download a character and/or animations from e.g. Mixamo for your Project, make sure to use these settings:
- Character *or* Base Anim:
  - FBX Binary (.fbx)
  - With Skin
- All other Animations:
  - FBX Binary (.fbx)
  - Without Skin

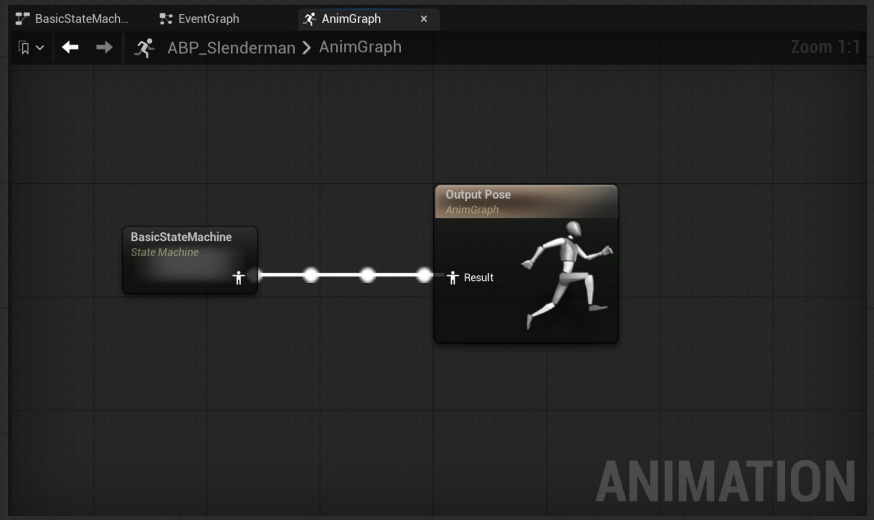
- To import Animations into the Engine, **first only import the character**, then import all other animations (because it first needs to create the Skeletal Mesh for your character!)



make sure to select the correct Skeleton when importing animations for your character!

Animation Blueprint



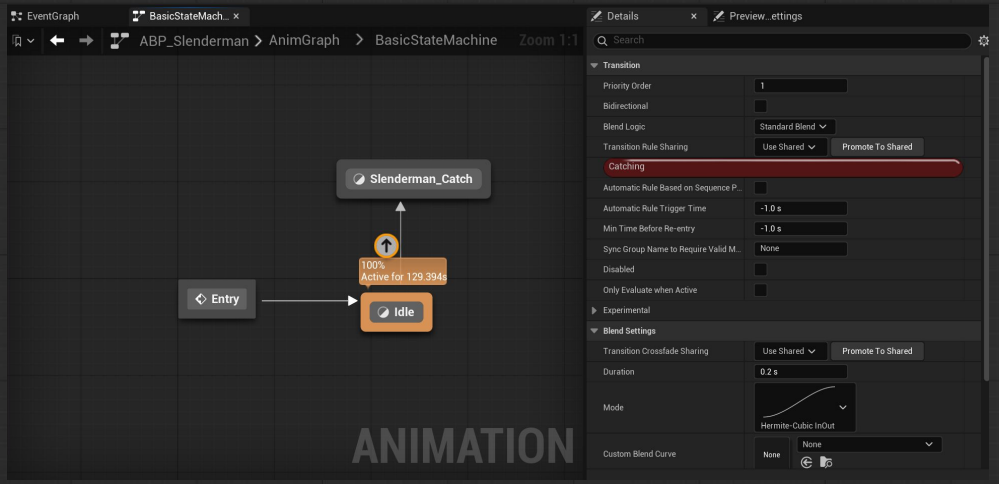


Animation Graph

- The Output Pose is the one we see in-game
- We can either use a simple animation, or we can create a State Machine

State Machine

- Composed of *States* and *Transitions*
- Starts at *Entry* Node
- Simply drag Animations inside to add States
- Click on Arrow to add Transition Rules (you need variables for that! They can be set in the Event Graph)
- We can also create Blend Spaces and add them instead of a single Animation



Assigning the Animation to the Pawn

- Once you created your Animation Blueprint, you can assign it to your Pawn
- You can find it either in the *Class Defaults*, or when clicking onto your *Character Mesh* Component

